Endterm Project

Course: Machine Learning (AMO 3208)

Instructor: Mimenbayeva A.B.

Topic: CNN Verifier for YOLOv8n Pedestrian Detections in a Danger-Zone Monitoring System

Group: SE-2405

Student: Bolatov Bekdaulet

05.11.2026

Code and data: github.com/BekdauletBol/ml-endterm-pedestrian-verifier

Table of Contents

1 Introduction
- 1.1 Problem Definition
- 1.2 Real-World Relevance
- 1.3 Objectives and Progress after Assignment 2

2 Literature Review

3 Dataset Description

4 Methodology
- 4.1 Data Preprocessing
- 4.2 Models
- 4.3 Training Setup
- 4.4 Three Hyperparameter Configurations
- 4.5 Improvement Techniques
- 4.6 Evaluation Metrics
- 4.7 Practical Application: Test on Real YOLOv8n Boxes

5 Results and Evaluation
- 5.1 Comparison of the Three Configurations
- 5.2 Effect of the Improvements
- 5.3 Best Model: Detailed Evaluation
- 5.4 Error Analysis
- 5.5 Practical Application Results

6 Discussion

7 Conclusion

References

1 Introduction

1.1 Problem Definition

This project is the last part of my Machine Learning course project, and it continues my **Assignment 2**. Everything is connected to my diploma project **public.safe**. In public.safe ordinary phones work as cameras. A laptop runs the **YOLOv8n** detector on the frames, finds people and checks if a person is inside a dangerous zone, for example a kerb or a loading area. If yes, the system creates a *zone_entered* event and the operator sees an alarm.

YOLOv8n is the smallest YOLOv8 model. It is fast, but it makes mistakes: sometimes it says "person" for a toy, a statue, an animal or a dark corner. Every wrong box inside a zone is a **false alarm**.

Problem type: supervised **image classification** with 3 classes. The input is the crop of one YOLOv8n box (128×128 RGB). The output is one of:

1. **pedestrian** – a real person;
2. **other_object** – an object that is not a person (car, chair, dog, bag, …);
3. **background** – an empty place without an object (road, wall, grass).

Only boxes that my model calls *pedestrian* go to the zone rules of public.safe. I call this model a **verifier**, because it checks (verifies) the boxes of the detector.

1.2 Real-World Relevance

- **Safety.** Automatic monitoring of dangerous places can help to protect pedestrians and workers, but only if people trust the alarms. If the system gives a false alarm every few minutes, the staff start to ignore it.
- **Cheap hardware.** public.safe must work on a normal laptop and phones, so I cannot simply take a much bigger detector. A small second model that looks only at the boxes is much cheaper than a big detector for the whole frame.
- **Same problem in research.** Many papers about danger-zone systems use YOLO boxes directly with geometric rules and do not check if the box is really a person (see Section 2). Papers about pedestrian detection show that a second-stage check can remove many false positives.

1.3 Objectives and Progress after Assignment 2

In Assignment 2 I defined the problem, reviewed the literature and built the dataset *PedCrops v1*. In this endterm I:

1. train my own CNN with the **three hyperparameter configurations** from the task and compare them;
2. improve the model with **BatchNorm, Dropout, data augmentation** and **transfer learning (MobileNetV2)**;

3. evaluate all models on the test set with accuracy, precision, recall, F1, confusion matrices and curves;
4. test the best model in a **practical scenario**: on real YOLOv8n boxes from the test photos, with the same detector weights that public.safe uses;
5. write a drop-in detector class (`VerifiedDetector`) with the same `detect()` method as the detector of public.safe.

2 Literature Review

For Assignment 2 I reviewed 12 publications from 2021–2025 about pedestrian detection, light YOLO models, danger-zone intrusion detection and false-positive reduction. Table 1 is a short version of that review.

Authors, year	Data	Model	Main result	Key finding / limitation
Xiao & Feng, 2023	CrowdHuman, MOT17/20	YOLOv8n + Ghost convs, OC-SORT	mAP@0.5 79.3 %, −40 % parameters	smaller model, but lower recall
Liu et al., 2024	WiderPerson + own images	YOLOv8-CB (YOLOv8n + attention)	mAP@0.5 54.7 % (YOLOv8n 52.3 %)	+2.4 mAP; speed only on a big GPU
Zhou et al., 2022	WiderPerson, CityPersons	YOLOv4-TP-Tiny	AP 58.3 % at 31 FPS	pedestrian attention helps on edge GPUs
Yan et al., 2022	own road-camera images	pruned YOLOv5s	mAP@0.5 83.4 %, 55.5 → 7.7 MB	extra class "others" for person-like objects
Mei et al., 2023	construction-site video	YOLOv5 + rules	precision 96.1 %, recall 90.1 %	no check that a box is really a person
Tran et al., 2023	own site videos	YOLOv8 + 4D-BIM zones	precision 95.3 %, recall 83.7 %	zones change over time automatically
Feng et al., 2024	Roboflow, 103,500 images	YOLOv8s	mAP@0.5 84 %, recall 60.5 %	low recall
Khan et al., 2022	CityPersons, Caltech, ECP	F2DNet	MR^{-2} 8.7 % CityPersons	a light head that only removes false positives
Hasan et al., 2022	5 pedestrian datasets	Cascade R-CNN	MR^{-2} 12.8 % cross-dataset	diverse web data (COCO-like) generalizes better
Apurv et al., 2021	own car recordings	YOLOv3 → MobileNetV2 on crops	accuracy 0.91, F1 0.91	closest to my pipeline; binary task, private data
Jiang et al., 2024	Caltech, CityPersons	proposals → recognition network	false positives 13,969 → 4,392	second network removes ~70 % of FPs
Guo et al., 2025	Caltech, CityPersons, SY-Metro	MetroNext + PSTNet	MR 24.68 % on SY-Metro	better training reduces FPs; slow on Jetson

Table 1. Reviewed works (short version of the Assignment 2 review).

What I took from these works:

- **Two-stage pipelines work.** Apurv et al. (2021) used exactly the design I use: YOLO finds people, and a fine-tuned MobileNetV2 classifies each crop (accuracy 0.91). Jiang et al. (2024) and Khan et al. (2022) show that a separate second stage can remove many false positives. This is why I chose **MobileNetV2** for transfer learning: it was designed for phones and it is already proven for crop classification.
- **Danger-zone systems trust the detector.** Mei et al. (2023), Tran et al. (2023) and Feng et al. (2024) put rules on top of YOLO boxes. None of them checks whether a box is really a person, and Yan et al. (2022) even needed an extra class for objects that look like pedestrians.
- **Data diversity matters.** Hasan et al. (2022) found that detectors trained on one driving dataset generalize badly, while diverse web images like COCO generalize better. This supports my

choice of COCO.

Research gap. I did not find a work that adds a light crop verifier to a phone-camera danger-zone system and measures the effect on the boxes of the same YOLOv8n model that the system uses. This project does that.

3 Dataset Description

Source. COCO val2017 (Lin et al., 2014): 4,952 photos with 36,781 human-made boxes, 11,004 of them persons. I downloaded it from the Hugging Face mirror `detection-datasets/coco`, because the official server was too slow for me. The COCO annotations are under the CC BY 4.0 license.

How the crops were made (Assignment 2, `data/build_dataset.py`):

- **pedestrian** – COCO person boxes with height ≥ 48 px and width ≥ 16 px (smaller people are too small to classify from a crop);
- **other_object** – boxes of the other 79 COCO classes with the shorter side ≥ 32 px; objects that overlap a person (IoU > 0.1 or more than 20 % covered by a person) are removed, so that a chair with a sitting person is not labelled “other_object”;
- **background** – random windows with a person-like shape (height/width 0.3–0.7) that do not overlap any person (IoU < 0.05) and almost no object (IoU < 0.3).

Every crop gets 10 % context around the box, is padded to a square with gray color (114, like YOLO) and resized to 128×128. Duplicates were removed with an average hash (1,074 crops), and the classes were balanced to **7,223 crops each**. The split is **70 / 15 / 15 by photo**, so all crops of one photo are in one split and there is no data leakage between training and test.

Class	Train	Validation	Test	Total
pedestrian	5,163	1,140	920	7,223
other_object	5,019	1,132	1,072	7,223
background	5,078	1,052	1,093	7,223
Total (crops)	**15,260**	**3,324**	**3,085**	**21,669**
Photos (41 photos gave no crops)	3,437	736	738	4,911

Table 2. PedCrops v1: number of crops per class and split.

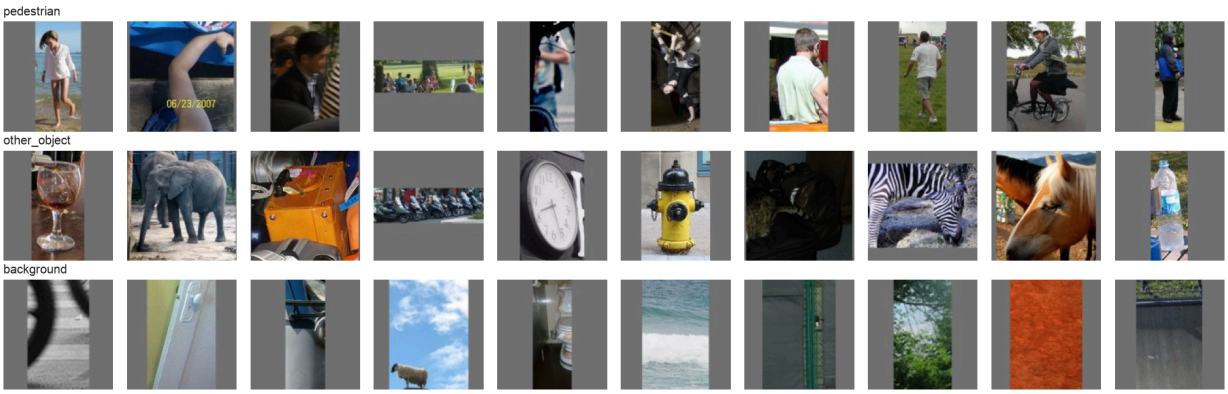


Figure 1. Random examples of the three classes (10 per class).

Second test set – real YOLOv8n boxes. For the practical part I also ran YOLOv8n on the **738 test photos** (`scripts/make_yolo_detections.py`). It found 1,184 "person" boxes. I compared every box with the COCO labels: a box with IoU ≥ 0.5 with a real person is a true positive (939 boxes), all others are false positives (245 boxes). These boxes were cropped in exactly the same way as the training crops.

4 Methodology

All code is written in **PyTorch** (Paszke et al., 2019). I trained on my laptop (Apple M1, 8 GB) with the `mps` GPU backend. The notebook is also prepared for Google Colab: its first cell clones the repository with the data and the weights.

4.1 Data Preprocessing

- **Resize.** The crops are stored at 128×128. My own CNN uses **64×64** input, so that I could train all configurations on my laptop in reasonable time. MobileNetV2 uses **128×128**.
- **Normalization.** For my CNN I use the mean and standard deviation of the training crops (mean 0.448 / 0.427 / 0.404, std 0.206 / 0.200 / 0.207). For MobileNetV2 I use the ImageNet values (0.485 / 0.456 / 0.406 and 0.229 / 0.224 / 0.225), because its pretrained weights expect them.
- **Augmentation (training only).** Random horizontal flip, rotation ±15°, scaling 0.85–1.15, shift ±10 %, brightness and contrast ±20 %. I do it on the GPU for every batch, so each epoch sees new versions of the images. Validation and test images are never augmented.
- **Split.** 70 / 15 / 15 by photo (Section 3). The test set is used only once, for the final evaluation.

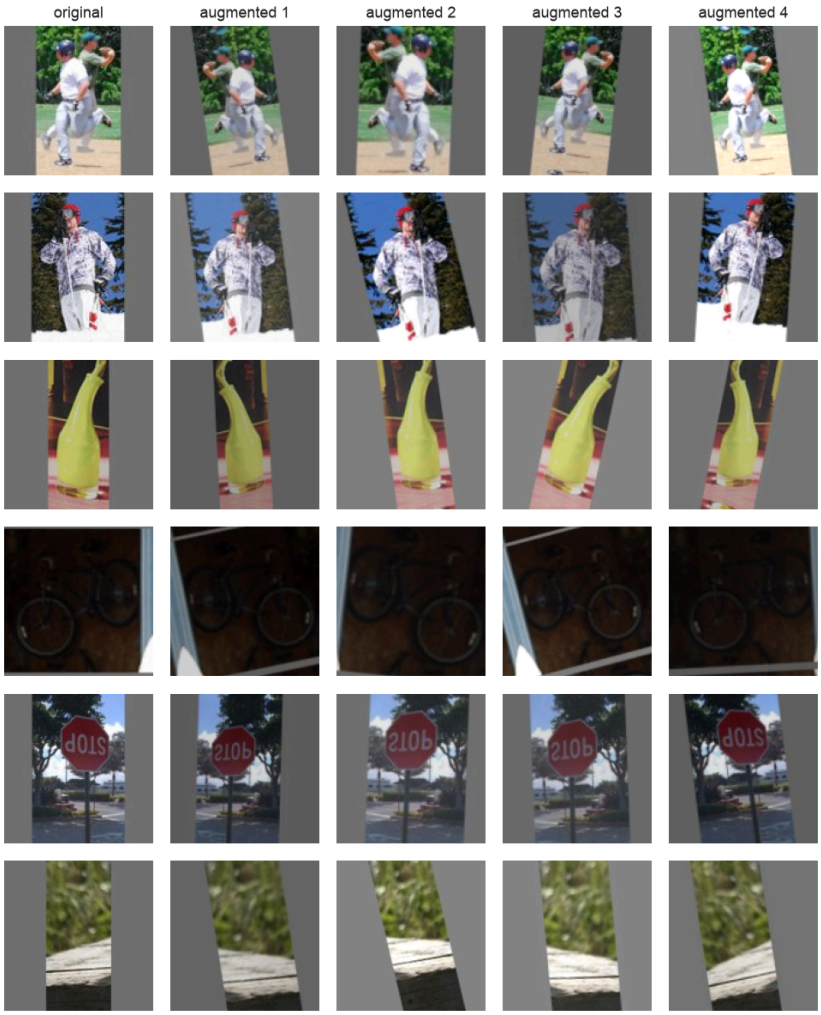


Figure 2. Data augmentation: two crops of each class (rows), the original on the left and four random augmented versions.

4.2 Models

Model A – my own CNN (`PedCNN`), trained from scratch. Four convolution blocks: 3×3 convolution → activation → 2×2 max-pooling, with 32, 64, 128 and 128 filters. Then global average pooling and a dense head: 128 → *hidden layer* → 3. The activation, the hidden size, the number of hidden layers and the weight initialization can be changed, so I can run the configurations of the task. The activation is used in the convolution blocks and in the hidden layer. The model has about **249 k parameters**.

Model B – MobileNetV2 with transfer learning (Sandler et al., 2018). The network is pretrained on ImageNet. I replaced the last layer with Dropout(0.2) + Linear(1280 → 3). The model has **2.23 M parameters**.

4.3 Training Setup

- **Loss:** cross-entropy. **Validation:** after every epoch on the validation set.
- **Early stopping:** if the validation loss does not improve for 6 epochs (4 for the MobileNetV2 fine-tuning), training stops, and the weights of the best epoch are restored.

- **Gradient accumulation.** A batch of 1,024 images does not fit into 8 GB of memory, so I process it in parts of 256 images and add up the gradients. The optimizer still makes one step per 1,024 images, so it is the same experiment as with a real batch of 1,024.

4.4 Three Hyperparameter Configurations

As required by the task, I trained my CNN with three sets of hyperparameters. All three use Adam with learning rate 0.001 (Kingma & Ba, 2015), Xavier-uniform initialization (Glorot & Bengio, 2010), one hidden layer, no augmentation and at most 30 epochs.

Configuration	Optimizer	Initialization	Activation	Hidden layer size	Batch size	Hidden layers
Config 1	Adam	XavierUniform	tanh	64	1024	1
Config 2	Adam	XavierUniform	tanh	32	128	1
Config 3	Adam	XavierUniform	relu	32	1024	1

Table 3. The three hyperparameter configurations.

4.5 Improvement Techniques

1. **Improved CNN.** ReLU, batch 128, hidden size 64, **BatchNorm** after every convolution (Ioffe & Szegedy, 2015), **Dropout 0.3** in the head (Srivastava et al., 2014), **data augmentation**, AdamW with learning rate 0.002 and weight decay 0.0001 (Loshchilov & Hutter, 2019) and a cosine learning-rate schedule (Loshchilov & Hutter, 2017).
2. **Transfer learning with MobileNetV2** in two stages, with augmentation, batch 64 and AdamW: *stage 1* – the pretrained backbone is frozen and only the new head is trained (3 epochs, lr 0.001); *stage 2* – the whole network is fine-tuned with a small learning rate (up to 10 epochs, lr 0.0001, cosine schedule).

4.6 Evaluation Metrics

- **Accuracy** – share of correct crops.
- **Precision, recall and F1** per class and as a macro average (all classes count equally). For public.safe the most important numbers are the **precision and recall of the pedestrian class**: low precision means false alarms, low recall means missed people.
- **Confusion matrix** – which classes are mixed up.
- **Loss and accuracy curves** for training and validation – to see underfitting and overfitting.

4.7 Practical Application: Test on Real YOLOv8n Boxes

The real question is not “how good is the model on my crops”, but “does it help YOLOv8n”. So I took the **same YOLOv8n weights as in public.safe** (I checked the SHA-256 checksum) and the same confidence threshold 0.30, and ran them on the 738 test photos. The verifier looks at every box and keeps it only if **P(pedestrian) ≥ threshold**. A good filter removes many false positives and keeps almost all true ones.

I also split the false positives into two types: **“part of a person”** (the box overlaps a real person with IoU between 0.3 and 0.5, for example a box on half of a body) – 114 boxes; and **“no person”** (IoU < 0.3 with every labelled person) – 131 boxes.

Finally I wrote `src/verified_detector.py`. The class `VerifiedDetector` has the same `detect(image, conf)` method as the detector in public.safe, so the zone rules of the diploma system do not need any change.

5 Results and Evaluation

5.1 Comparison of the Three Configurations

Configuration	Accuracy (test)	Loss (test)	Training Time	Comments
Config 1 (tanh, 64, batch 1024)	0.805	0.466	19.2 min, 30 epochs	slow start (15 weight updates per epoch); still improving at epoch 30 – under-trained
Config 2 (tanh, 32, batch 128)	0.805	0.482	6.6 min, 18 epochs	learns fastest (120 updates per epoch); overfits after epoch 12 (train 96 %, validation 83 %); early stopping
Config 3 (relu, 32, batch 1024)	**0.808**	**0.463**	8.6 min, 30 epochs	almost the same curves as Config 1; relu slightly better; still improving at epoch 30

Table 4. Results of the three configurations on the test set (3,085 crops).

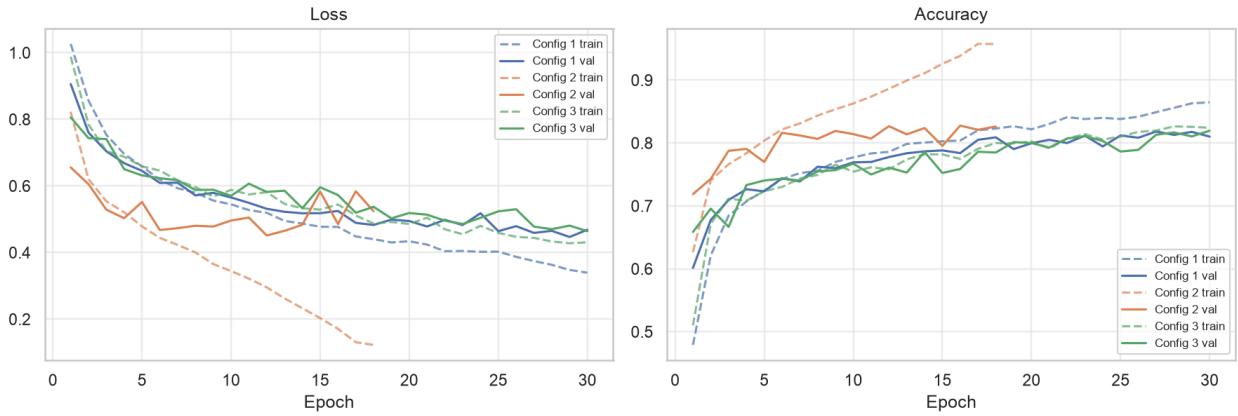


Figure 3. Loss and accuracy curves of the three configurations (dashed = training, solid = validation).

Analysis.

- **All three reach almost the same test accuracy (80.5–80.8 %).** The difference of 0.3 percentage points is about 10 of 3,085 test crops, so on the test set none of them is clearly better. With this small CNN and no augmentation, the limit is the model and the missing regularization, not these three hyperparameters.
- **The batch size changes *how* the model learns.** With 15,260 training images, batch 1,024 gives only 15 weight updates per epoch, batch 128 gives 120. Config 2 reached 79 % validation accuracy already in epoch 4; Config 1 needed 17 epochs for the same. But Config 2 also overfits quickly: its training accuracy went up to 96 % while the validation accuracy stayed at about 83 %, the validation loss was lowest in epoch 12, and early stopping ended the run in epoch 18. Configs 1 and 3 had their best epoch at 29–30, so they were still learning (under-trained, not overfitted). The validation curve of Config 2 is also more noisy, because a small batch gives a more noisy gradient.
- **Activation (Config 1 tanh vs Config 3 relu, both batch 1,024).** The curves are almost the same. ReLU is a little better (80.8 % vs 80.5 %, loss 0.463 vs 0.466). My network has only four convolution layers, and Xavier initialization keeps tanh in its linear range, so the vanishing-gradient problem of tanh is small here.

- **Hidden size (64 vs 32).** The head (128 → hidden → 3) has 8.5 k or 4.2 k weights out of about 249 k, so it changes almost nothing.
- **Training time.** All three configurations use the same part size of 256 images, so one epoch costs about the same. I timed two epochs of each again, one after another: after the first warm-up epoch it was 28–38 seconds per epoch for all three. So the different times per epoch in the original runs come from the state of my laptop, not from the hyperparameters. The total time mostly depends on the number of epochs: Config 2 was the fastest only because early stopping ended it after 18 epochs.

5.2 Effect of the Improvements

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	Pedestrian precision	Pedestrian recall	Loss	Epochs	Time
Config 1	0.805	0.803	0.803	0.803	0.722	0.779	0.466	30	19.2 min
Config 2	0.805	0.805	0.803	0.803	0.719	0.776	0.482	18	6.6 min
Config 3	0.808	0.809	0.808	0.804	0.705	0.811	0.463	30	8.6 min
Improved CNN	0.853	0.851	0.851	0.850	0.822	0.810	0.370	30	17.2 min
MobileNetV2, frozen backbone	0.838	0.843	0.838	0.839	0.878	0.842	0.422	3	4.2 min
MobileNetV2, fine-tuned	**0.928**	**0.928**	**0.929**	**0.928**	**0.925**	**0.941**	**0.214**	10	28.0 min

Table 5. All models on the test set. The fine-tuned MobileNetV2 starts from the frozen one, so its full training time is 4.2 + 28.0 = 32.2 min.

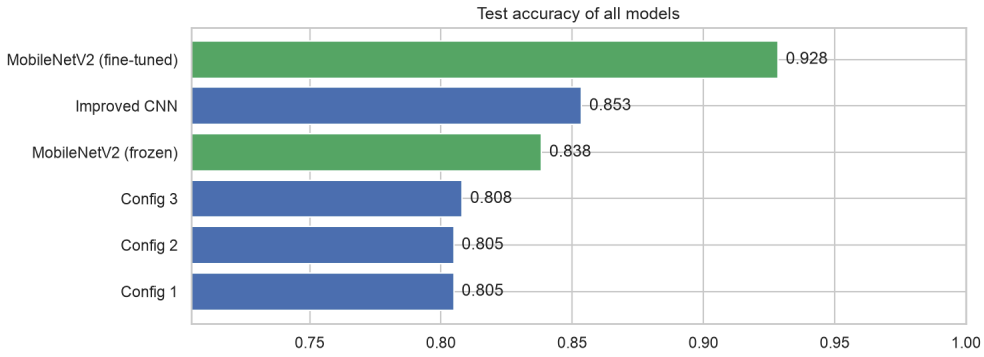


Figure 4. Test accuracy of all models.

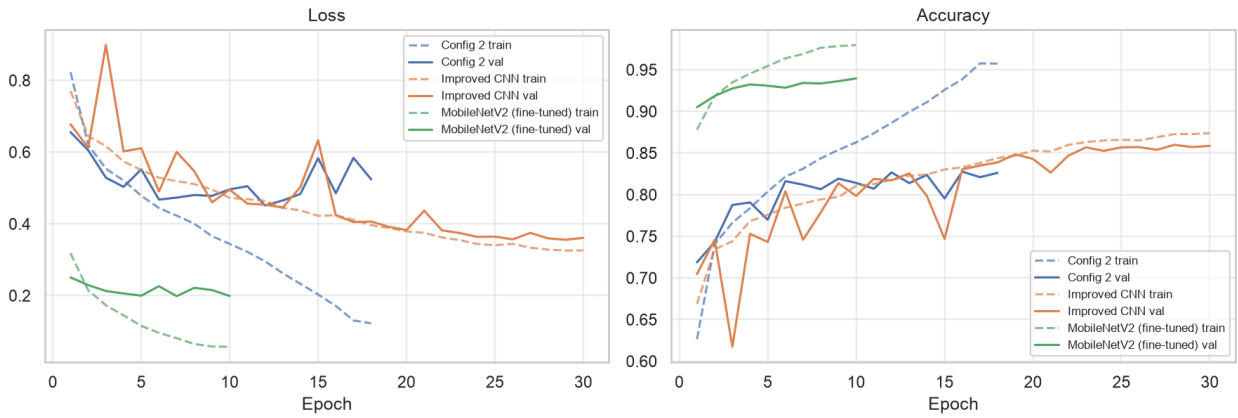


Figure 5. Curves of Config 2, the improved CNN and the fine-tuned MobileNetV2.

- **BatchNorm + Dropout + augmentation** raised the accuracy of my own CNN from 80.8 % to **85.3 %** and the pedestrian precision from 0.71–0.72 to 0.82. In Figure 5 the improved CNN does not overfit: training and validation accuracy stay close (87 % and 86 %), while Config 2 had a gap of 13 percentage points. Augmentation makes the training task harder (the training accuracy is lower), but the model generalizes better. The improved CNN still had its best epoch at 29 of 30, so a longer training could help a bit more.
- **Transfer learning** gave the biggest step. Only training the new head (frozen backbone) already gave 83.8 % after 3 epochs. Fine-tuning the whole network gave **92.8 %**, which is 7.5 percentage points better than my best own CNN. It started at 90.5 % validation accuracy in the first epoch, because the ImageNet features already know shapes of people and objects. After epoch 7 the validation loss stopped improving while the training accuracy grew to 98 %, so early stopping restored the weights of epoch 7.
- The price is speed: MobileNetV2 is about 9 times bigger, and one epoch took almost 3 minutes on my laptop (my CNN: about 30 seconds).

5.3 Best Model: Detailed Evaluation

Class	Precision	Recall	F1-score	Support
pedestrian	0.925	0.941	0.933	920
other_object	0.924	0.899	0.912	1,072
background	0.935	0.946	0.940	1,093
macro average	**0.928**	**0.929**	**0.928**	3,085

Table 6. Per-class metrics of the fine-tuned MobileNetV2 on the test set (accuracy 0.928).

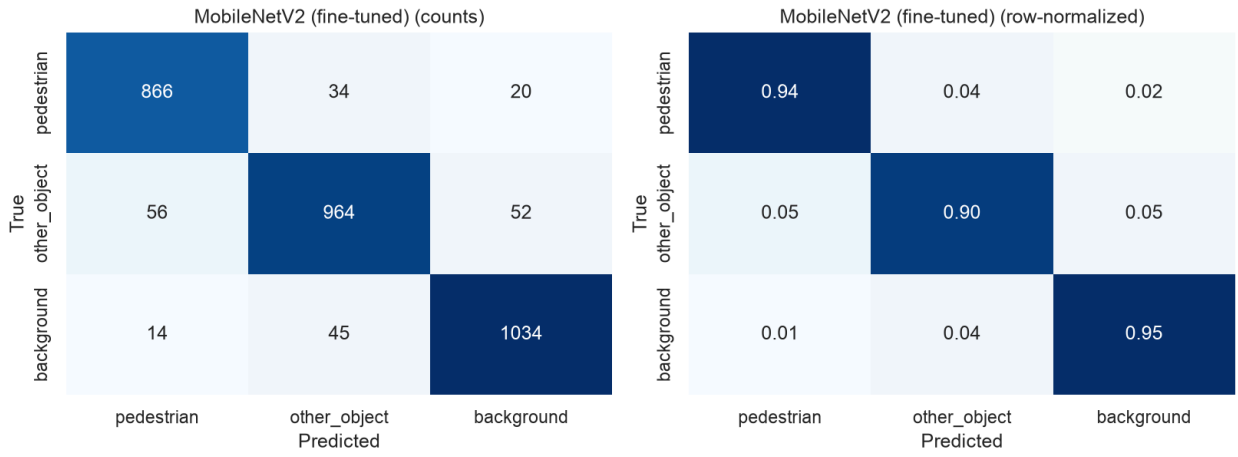


Figure 6. Confusion matrix of the fine-tuned MobileNetV2 (left: counts, right: share of each true class).

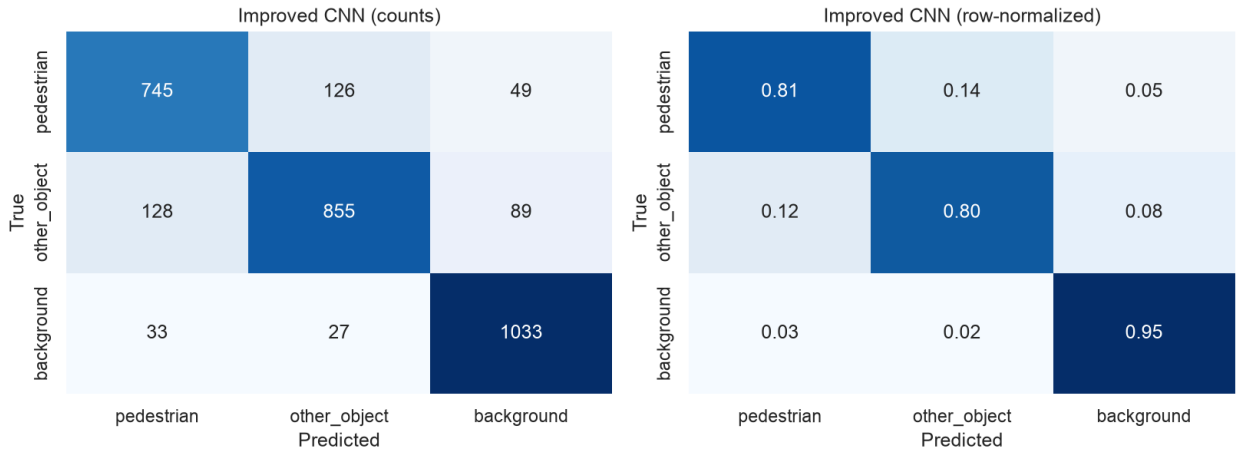


Figure 7. Confusion matrix of my best own CNN (improved CNN), for comparison.

- **Pedestrian** is the easiest class to find: 94 % of the real pedestrians are recognized.
- **other_object** is the hardest class (recall 0.90): 56 objects were called pedestrian and 52 were called background. COCO objects are very different (from a fork to a train), and some small objects look like background.
- **For public.safe** two numbers matter. 70 of 2,165 non-pedestrian crops (3.2 %) were called pedestrian; they would still give false alarms. 54 of 920 pedestrians (5.9 %) were called something else; they would be missed.

5.4 Error Analysis

The best model made 221 mistakes on 3,085 test crops. Figure 8 shows 16 random mistakes.

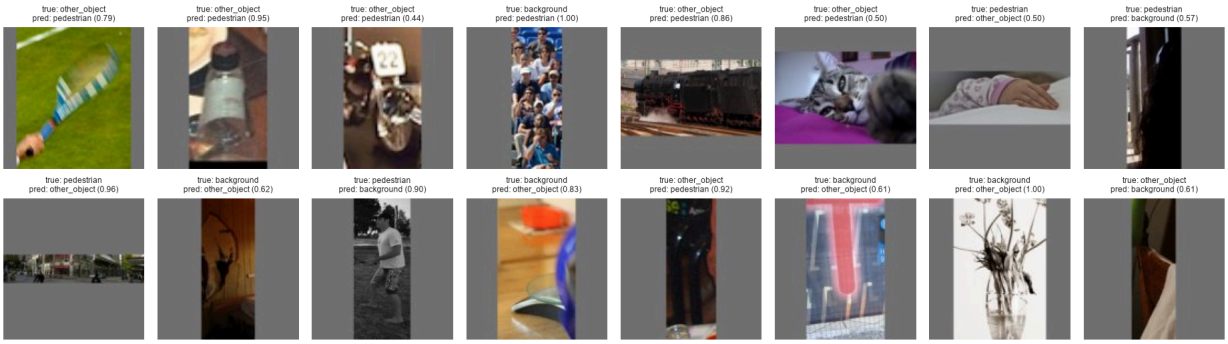


Figure 8. Random mistakes of the fine-tuned MobileNetV2 (true class, predicted class and probability).

- **Some labels are not really correct.** For example, a "background" crop full of spectators (4th crop in the first row) was predicted as pedestrian with probability 1.00. The people in it simply have no COCO boxes, so my rule for background windows did not see them. A bat held by a hand (1st crop) contains a part of a person. Here the model is closer to the truth than the label.
- **Objects near people.** The other_object mistakes come most often from *horse* (10), *chair* (9), *potted plant* (8) and *umbrella* (8). Horses, chairs and umbrellas are very often used by people in COCO photos, so parts of people can be inside these crops even after my overlap filter.
- **Hard crops.** Very dark, blurred or tiny crops, and people seen from an unusual angle (a hand on a bed, a person in a dark doorway).

5.5 Practical Application Results

YOLOv8n alone, on the 738 test photos: 1,184 boxes, 939 true and 245 false. Precision 0.793 and recall 0.603 (the recall is low because COCO also labels very small and hidden people). Table 7 shows what happens when the verifier filters these boxes.

Threshold on P(pedestrian)	True pedestrians kept	False positives left	FP removed	Precision	Recall
0 (YOLOv8n only)	939 (100 %)	245	0 %	0.793	0.603
0.3	910 (96.9 %)	220	10.2 %	0.805	0.584
0.5	**903 (96.2 %)**	**213**	**13.1 %**	**0.809**	**0.580**
0.7	889 (94.7 %)	206	15.9 %	0.812	0.571
0.9	856 (91.2 %)	190	22.4 %	0.818	0.550

Table 7. YOLOv8n + verifier on the 738 test photos.

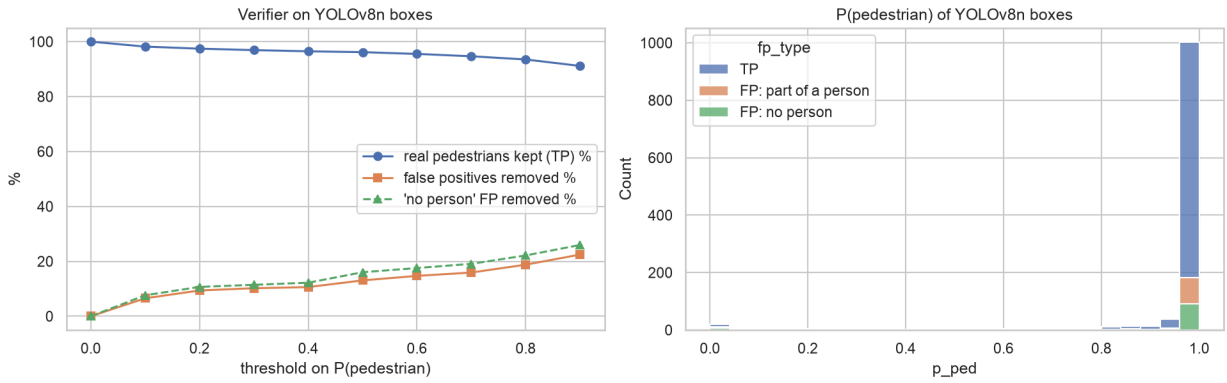


Figure 9. Left: true pedestrians kept and false positives removed for different thresholds. Right: P(pedestrian) of all YOLOv8n boxes.

At my default threshold 0.5 the verifier removes **13 % of the false positives** (245 → 213) and loses **3.8 % of the true pedestrians** (939 → 903). The precision goes from 0.793 to 0.809. On my laptop the verifier needs **32 ms for 16 boxes**, about **2 ms per box**, so it is fast enough for public.safe.

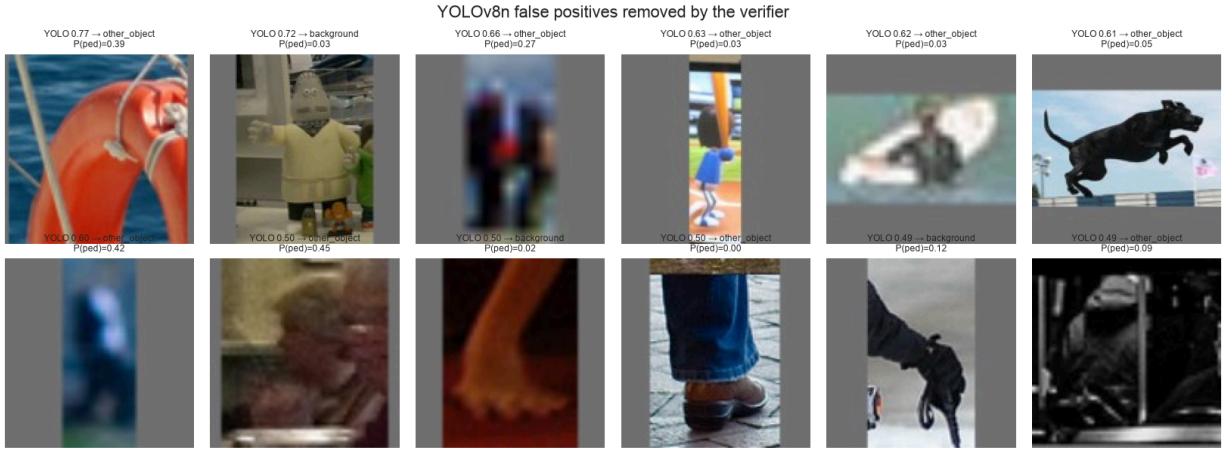


Figure 10. False positives of YOLOv8n that the verifier removed (YOLO confidence, predicted class, P(pedestrian)).

Most removed boxes (Figure 10) are what I wanted to remove: a life buoy, a statue, a toy figure, a dog and a boat, and also boxes that contain only a foot, a leg or a hand. A few of them look like blurred people, so not every removed box is a real mistake of YOLOv8n.

Why is the effect so small? This result was smaller than I expected, so I checked the boxes. Figure 9 (right) shows that most false positives get P(pedestrian) close to 1, just like the true ones. Two reasons:

1. **114 of the 245 false positives are "part of a person"**: the box overlaps a real person, but not enough for IoU ≥ 0.5 (for example a box on the upper body only, or two boxes on one person). There really is a person in these crops, so the verifier correctly says pedestrian. For a danger zone this is not even a false alarm, because a person is really there.
2. **Many "no person" boxes are people without a COCO label.** The verifier kept 110 of the 131 "no person" boxes. I looked at 40 random ones of them (Figure 11) and counted by hand:

What the crop shows	Count
a real person or a part of a person **without a COCO box** (spectators, people in a crowd, people at the image border)	27
not a person – a real mistake (toy figure, dog, bronze statue, fire hydrant, donkey with a cart)	5
too small, dark or blurred to decide	8

Table 8. My manual check of 40 "no person" boxes that the verifier kept.

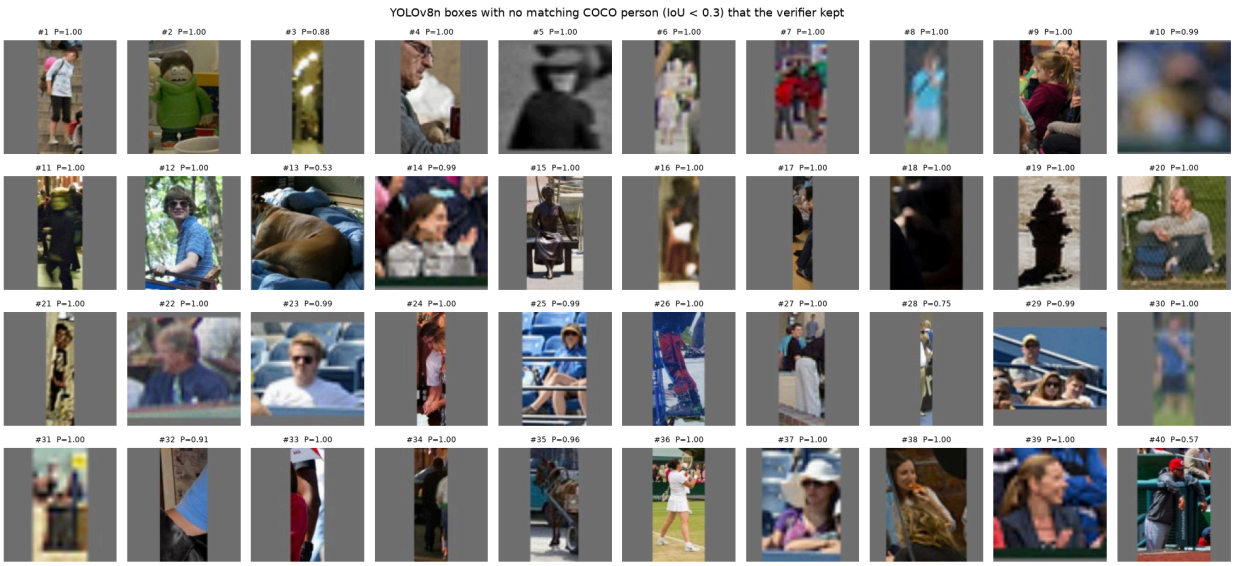


Figure 11. The 40 checked boxes: YOLOv8n "persons" with no matching COCO person that the verifier kept.

So for about two thirds of the kept "false positives" the verifier is right and the COCO labels are incomplete. If the same share is true for all 110 boxes, about 74 of them are real people. The real false positives – toys, statues, animals – are only a small part of the 245 boxes, and this is why the measured reduction is only 13 %. The numbers in Table 7 are therefore a **pessimistic** estimate.

The lost pedestrians. I also looked at the 36 true pedestrians that the verifier rejected. Many of them are only partly visible (only hands or legs, a person in the dark), 8 are smaller than 48 px (smaller than any pedestrian in my training set), and some are in unusual poses (lying, skiing). But a few are clear people, for example baseball players behind a net, so the model still has real misses.

6 Discussion

What worked. Transfer learning was the most effective technique: 92.8 % accuracy against 85.3 % for my best own CNN. Regularization (BatchNorm, Dropout) and augmentation together gave +4.5 percentage points to my own CNN and removed the overfitting. Of the three hyperparameter sets, the batch size had the biggest influence: it decided how fast the model learned and how fast it overfitted. The activation and the hidden size changed the result very little in this architecture.

Comparison with the literature. Apurv et al. (2021) reached accuracy 0.91 with a MobileNetV2 crop classifier for a 2-class task. My MobileNetV2 reaches 0.928 on 3 classes, so the result is at a

similar level. Jiang et al. (2024) removed about 70 % of the false positives with a second-stage network on Caltech; my verifier removed 13 % of the COCO false positives of YOLOv8n. As Section 5.5 shows, the main reason is that most "false positives" of YOLOv8n on COCO are real people or parts of people. The results are therefore not directly comparable.

Limitations.

- **Label noise in COCO.** Not every person in COCO has a box. This affects the background class of my dataset and, even more, the evaluation of the practical test.
- **Domain.** COCO photos are not frames from phone cameras at a kerb or a loading area. The real false positives of public.safe can be different (poles, bags, shadows), and I could not test this without labelled frames from my own cameras.
- **One run per model.** I trained each model once with one random seed. Small differences such as 0.3 percentage points between the configurations are not reliable.
- **Small input for my CNN.** My own CNN used 64×64 images to save time; with 128×128 it could be better.
- **Partial people.** The verifier rejects some boxes that contain only a hand or a foot of a real person. In a danger zone such a box can still mean a real person, so it costs some recall (3.8 % at threshold 0.5).

Future work. Record and label frames from my own phone cameras and fine-tune the verifier on them; clean the COCO labels (add boxes for the unlabelled people); use the verifier only for boxes inside zones; combine it with tracking across frames; add `VerifiedDetector` to public.safe and measure false alarms per hour in a real test.

7 Conclusion

In this endterm project I continued my Assignment 2 and built a CNN verifier for the YOLOv8n person boxes of my diploma system public.safe.

- On my dataset *PedCrops v1* (21,669 crops from COCO, 3 balanced classes, split by photo) I trained my own CNN with the three required hyperparameter configurations. They reached 80.5–80.8 % test accuracy. The batch size changed the learning speed and the overfitting the most.
- BatchNorm, Dropout and augmentation improved my own CNN to 85.3 %. **Transfer learning with MobileNetV2** gave the best model: **92.8 % accuracy, macro F1 0.928, pedestrian recall 94.1 %**.
- On **real YOLOv8n boxes** the verifier (threshold 0.5) removed 13 % of the false positives and kept 96.2 % of the true pedestrians, with about 2 ms per box. My manual check showed that most remaining "false positives" are real people without a COCO label, so the real effect is larger than the measured one. Still, the effect has to be measured on frames from the real cameras before I switch the verifier on in public.safe.

- The verifier is ready to use as `VerifiedDetector` with the same interface as the public.safe detector.

Compared with Assignment 2, where I only prepared the data, the project now has trained and evaluated models, a clear comparison of hyperparameters and improvement techniques, and a test in the real pipeline of my diploma project.

References

Apurv, K., Tian, R., & Sherony, R. (2021). *Detection of e-scooter riders in naturalistic scenes* (arXiv:2111.14060). https://arxiv.org/abs/2111.14060

Feng, R., Miao, Y., & Zheng, J. (2024). A YOLO-based intelligent detection algorithm for risk assessment of construction sites. *Journal of Intelligent Construction, 2*(4), 9180037. https://doi.org/10.26599/JIC.2024.9180037

Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics* (pp. 249–256). PMLR.

Guo, Q., Zhang, R., & Zhang, B. (2025). A pedestrian sensitive training algorithm for false positives suppression in two-stage CNN detection methods. *Computers, Materials & Continua, 84*(1), 1307–1327. https://doi.org/10.32604/cmc.2025.063288

Hasan, I., Liao, S., Li, J., Akram, S. U., & Shao, L. (2022). *Pedestrian detection: Domain generalization, CNNs, transformers and beyond* (arXiv:2201.03176). https://arxiv.org/abs/2201.03176

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning* (pp. 448–456). PMLR.

Jiang, Z., Huang, S., & Li, M. (2024). A pedestrian detection network based on an attention mechanism and pose information. *Applied Sciences, 14*(18), 8214. https://doi.org/10.3390/app14188214

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8* (Version 8.0.0) [Computer software]. https://github.com/ultralytics/ultralytics

Khan, A. H., Munir, M., van Elst, L., & Dengel, A. (2022). F2DNet: Fast focal detection network for pedestrian detection. In *2022 26th International Conference on Pattern Recognition (ICPR)* (pp. 4658–4664). IEEE. https://doi.org/10.1109/ICPR56361.2022.9956732

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *Computer Vision – ECCV 2014* (pp. 740–755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48

Liu, Q., Ye, H., Wang, S., & Xu, Z. (2024). YOLOv8-CB: Dense pedestrian detection algorithm based on in-vehicle camera. *Electronics, 13*(1), 236. https://doi.org/10.3390/electronics13010236

Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. In *5th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1608.03983

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1711.05101

Mei, X., Zhou, X., Xu, F., & Zhang, Z. (2023). Human intrusion detection in static hazardous areas at construction sites: Deep learning–based method. *Journal of Construction Engineering and Management, 149*(1), 04022142. https://doi.org/10.1061/(ASCE)CO.1943-7862.0002409

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024–8035).

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). https://doi.org/10.1109/CVPR.2018.00474

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929–1958.

Tran, S. V.-T., Lee, D., Bao, Q. L., Yoo, T., Khan, M., Jo, J., & Park, C. (2023). A human detection approach for intrusion in hazardous areas using 4D-BIM-based spatial-temporal analysis and computer vision. *Buildings, 13*(9), 2313. https://doi.org/10.3390/buildings13092313

Xiao, X., & Feng, X. (2023). Multi-object pedestrian tracking using improved YOLOv8 and OC-SORT. *Sensors, 23*(20), 8439. https://doi.org/10.3390/s23208439

Yan, X., He, Z., Huang, Y., Xu, X., Wang, J., Zhou, X., Wang, C., & Lu, Z. (2022). A lightweight pedestrian intrusion detection and warning method for intelligent traffic security. *KSII Transactions on Internet and Information Systems, 16*(12), 3904–3922. https://doi.org/10.3837/tiis.2022.12.007

Zhou, H., Wu, T., Sun, K., & Zhang, C. (2022). Towards high accuracy pedestrian detection on edge GPUs. *Sensors, 22*(16), 5980. https://doi.org/10.3390/s22165980